Ngram Prediction

1. Import Library

```
from collections import defaultdict
```

Purpose

defaultdict is a special dictionary that automatically creates empty lists for new keys.

Example:

```
bigrams = defaultdict(list)
bigrams["love"].append("machine")
```

No need to initialize the list first.

2. Define the Corpus

```
corpus = """
i love machine learning
i love deep learning
i study machine learning
machine learning is interesting
"""
```

Purpose

This corpus acts as the training data.

Sentences:

1. i love machine learning
2. i love deep learning
3. i study machine learning
4. machine learning is interesting

3. Tokenization

```
words = corpus.lower().split()
```

What Happens?

- Converts all text to lowercase.
- Splits the corpus into words.

Result:

```
[
'i', 'love', 'machine', 'learning',
'i', 'love', 'deep', 'learning',
'i', 'study', 'machine', 'learning',
'machine', 'learning', 'is', 'interesting'
]
```

4. Build Bigram Dictionary

```
bigrams = defaultdict(list)
```

Creates an empty dictionary where:

```
word → [possible next words]
```

Example format:

```
{
  "i": [...],
  "love": [...],
  "machine": [...]
}
```

5. Generate Bigrams

```
for i in range(len(words)-1):
```

Loops through all words except the last one.

For each position:

```
current_word = words[i]
next_word = words[i+1]
```

Example:

```
i → love
love → machine
machine → learning
```

6. Store Word Relationships

```
bigrams[current_word].append(next_word)
```

This stores all possible next words.

After Processing

The dictionary becomes approximately:

```
{
 'i': ['love', 'love', 'study'],
 'love': ['machine', 'deep'],
 'machine': ['learning', 'learning', 'learning'],
 'learning': ['i', 'i', 'machine', 'is'],
 'study': ['machine'],
 'deep': ['learning'],
 'is': ['interesting']
}
```

7. Get User Input

```
input_word = input("Enter a word: ")
```

Example input:

```
machine
```

8. Check Whether Word Exists

```
if input_word in bigrams:
```

Checks if the entered word appears in the corpus.

Example:

```
"machine" in bigrams
```

returns:

```
True
```

9. Predict Most Frequent Next Word

```
prediction = max(
    set(bigrams[input_word]),
    key=bigrams[input_word].count
)
```

This is the most important part.

Step 1

For input:

```
machine
```

Possible next words:

```
['learning', 'learning', 'learning']
```

Step 2

Convert to unique values:

```
set(...)
```

Result:

```
{'learning'}
```

Step 3

Find the word occurring most frequently:

```
max(..., key=count)
```

Result:

```
learning
```

Thus:

```
Predicted next word: learning
```

Example Predictions

Input

```
i
```

Next words:

```
['love', 'love', 'study']
```

Frequency:

Word	Count
love	2
study	1

Prediction:

```
love
```

Input

```
love
```

Next words:

```
['machine', 'deep']
```

Both occur once.

The program returns one of the highest-frequency words.

Output Example

Input

```
Enter a word: machine
```

Output

```
Predicted next word: learning
```

What NLP Concept Is Demonstrated?

This program implements a **Bigram Language Model**, where:

Meaning:

> Predict the next word based on the previous word.

Examples:

- machine → learning
- study → machine
- is → interesting

Limitation of This Model

The model:

- Uses only one previous word.
- Cannot understand meaning.
- Returns only the most frequent next word.
- Gives no prediction for unseen words.

Example:

```
Input: artificial
```

Output:

```
Word not found in corpus
```

Real-World Connection

This simple bigram predictor is the foundation of:

- Mobile keyboard suggestions
- Text autocomplete
- Search query prediction
- Speech recognition
- Statistical language models

Modern AI systems such as large language models extend this idea by considering much larger contexts instead of just one previous word.

Time Complexity

- Tokenization: **O(N)**
- Bigram Construction: **O(N)**
- Prediction: **O(k)**, where *k* is the number of possible next words for the input word.

Overall training complexity: **O(N)**.